

Assignment1

September 17, 2026

0.0.1 Applied Comparative Genomics: Assignment 1

Ashrith Kollu

akollu1

akollu1@jh.edu

0.0.2 Question 1: Why Genomics?

- Question 1.1. Use ChatGPT (or your favorite LLM) to write an essay on why you are interested in genomics. Include specific details from your own interests. Make sure to ask for references. Make sure to include both your prompt(s) and the output from the LLM.

Prompt:

Write an essay on why I'm interested in genomics. Be sure to include how it has the potential to revolutionize medicine by personalizing it to target disease sources. Focus on chronic genetic illnesses with no good treatment options. Bring in recent examples of breakthroughs that have happened and their effect in healthcare. Be sure to cite your sources and have them in AMA format.

LLM Output:

I am interested in genomics because it offers a way to understand disease at its source. Rather than treating only symptoms after they appear, genomics examines the DNA variants, gene-expression patterns, and molecular pathways that cause illness in the first place. This approach has the potential to revolutionize medicine by making care more precise: treatment can be selected, designed, or adjusted according to the biological features of an individual patient's disease.

This possibility is especially meaningful for people with chronic genetic illnesses that have historically had few effective treatment options. Many inherited disorders begin early in life, persist indefinitely, and affect multiple organ systems. Until recently, patients with conditions such as sickle cell disease, cystic fibrosis, spinal muscular atrophy, and ultra-rare neurologic disorders often relied on supportive care—blood transfusions, pain control, physical therapy, surgery, or management of complications. These measures can be lifesaving, but they do not correct the underlying genetic cause. Genomics changes that equation by identifying the specific mutation or pathway responsible for disease and creating an opportunity to intervene more directly.

Sickle cell disease is one of the clearest recent examples. The condition is caused by a change in the HBB gene that produces abnormal hemoglobin. Red blood cells become rigid and sickle-shaped, leading to severe pain episodes, anemia, strokes, organ damage, and shortened life expectancy. For many patients, treatment has focused on preventing complications rather than eliminating the

source of the disease. In December 2023, the US Food and Drug Administration approved exagamglogene autotemcel, marketed as Casgevy, the first FDA-approved therapy using CRISPR/Cas9 gene editing for sickle cell disease.¹ The treatment edits a patient's own blood-forming stem cells so that they produce more fetal hemoglobin, which can prevent red blood cells from sickling. In a pivotal clinical trial, nearly all evaluable patients with recurrent vaso-occlusive crises remained free of severe crises for at least 12 consecutive months after treatment.²

This breakthrough is important not only because it offers a new option for sickle cell disease, but because it demonstrates that genomics can move medicine beyond disease management toward molecular correction. A single genetic diagnosis can now guide a treatment that changes how a patient's cells behave. Although gene-editing therapies currently require specialized centers, intensive chemotherapy, and substantial financial resources, their success establishes a model for future therapies targeting other inherited blood disorders.

Genomics has also transformed care for cystic fibrosis, a chronic disorder caused by variants in the CFTR gene. Cystic fibrosis affects the lungs, pancreas, digestive system, and reproductive organs. For decades, care focused largely on antibiotics, airway clearance, nutritional support, and eventually lung transplantation for some patients. The development of CFTR modulators showed how genomic information can personalize treatment. Elexacaftor-tezacaftor-ivacaftor, a combination therapy designed to improve the function of certain defective CFTR proteins, significantly improved lung function and reduced pulmonary exacerbations in patients with at least one F508del mutation, the most common disease-causing variant.³

The impact on healthcare has been substantial. For eligible patients, CFTR modulators have changed cystic fibrosis from a condition treated primarily through complication management to one in which the faulty protein itself can be targeted. However, this example also reveals an important challenge: not every patient has a mutation responsive to currently available modulators. Continued genomic research is necessary to ensure that rare variants and underserved patient populations are not left behind.

Another reason I am drawn to genomics is its potential to help children with devastating conditions for which there may be little time to act. Spinal muscular atrophy (SMA), for example, is caused by loss or dysfunction of the SMN1 gene. In its most severe form, infants develop progressive muscle weakness, difficulty swallowing, respiratory failure, and early death without treatment. Gene-replacement therapy with onasemnogene abeparvovec delivers a functional copy of SMN1 to motor neurons. Early clinical evidence showed that treated infants achieved motor milestones that would have been unlikely in the natural course of severe SMA.⁴ Along with other therapies such as nusinersen and risdiplam, genomic medicine has changed the outlook for many children diagnosed through newborn screening.

Perhaps the most compelling frontier is the development of treatments for extremely rare diseases—conditions so uncommon that traditional drug development may not be commercially viable. In 2019, researchers created milasen, a personalized antisense oligonucleotide therapy for a single patient with Batten disease caused by a unique mutation in MFSD8.⁵ The treatment was designed to alter RNA splicing and increase production of a more functional protein. Although this was not a cure, it represented a major step toward individualized medicines designed around one patient's genomic information. It showed that a genetic diagnosis can be more than an explanation for why a disease occurred; it can become a blueprint for treatment.

I am interested in genomics because it combines biology, technology, and compassion. It recognizes that diseases with the same name may not be biologically identical and that patients deserve

therapies designed for the mechanisms driving their illness. For people living with chronic genetic diseases that have long lacked good treatment options, genomics offers hope for earlier diagnosis, more accurate prognosis, fewer ineffective treatments, and therapies aimed at the root cause of disease.

At the same time, the field raises important ethical and practical questions. Gene-based therapies can be extraordinarily expensive, access to genetic testing remains unequal, and genomic data must be protected from misuse. These challenges make genomics not only a scientific field, but also a healthcare equity issue. I am interested in contributing to a future in which genomic discoveries are translated responsibly and made available to the patients who need them most. Ultimately, genomics has the power to make medicine more personal, more preventive, and more capable of treating diseases that once seemed untreatable.

References US Food and Drug Administration. FDA approves first gene therapies to treat patients with sickle cell disease. Published December 8, 2023. Accessed September 14, 2026. <https://www.fda.gov/news-events/press-announcements/fda-approves-first-gene-therapies-treat-patients-sickle-cell-disease>

Frangoul H, Altshuler D, Cappellini MD, et al. Exagamglogene autotemcel for severe sickle cell disease. *N Engl J Med*. 2024;390(18):1649-1662. doi:10.1056/NEJMoa2309676

Middleton PG, Mall MA, Dřevínek P, et al. Elexacaftor-tezacaftor-ivacaftor for cystic fibrosis with a single Phe508del allele. *N Engl J Med*. 2019;381(19):1809-1819. doi:10.1056/NEJMoa1908639

Mendell JR, Al-Zaidy S, Shell R, et al. Single-dose gene-replacement therapy for spinal muscular atrophy. *N Engl J Med*. 2017;377(18):1713-1722. doi:10.1056/NEJMoa1706198

Kim J, Hu C, Moufawad El Achkar C, et al. Patient-customized oligonucleotide therapy for a rare genetic disease. *N Engl J Med*. 2019;381(1):64-74. doi:10.1056/NEJMoa1813279

- Question 1.2. Comment on the output from the LLM - does it make logical sense, does it include any phrases you would not have written, do the cited papers exist and support the claims from the LLM?

The LLMs do make logical sense. The essay flows naturally while addressing the specifications of the prompt. The names of the drugs and all the examples given are real and link back to actual sources, surprisingly, that support the claims generated by the LLM. The writing style is drastically different than what I would type, but it is something that is good enough to submit. The main difference is the vocabulary choice; I would not write with such a diverse selection of words while conveying my thoughts.

- Question 1.3 Post your final prompt, the LLM's response, and which LLM you used to piazza. Include the link to the post in your submission https://piazza.com/class/mt9gq0k8adb717/post/11_f15

0.0.3 Question 2: Chromosome structures

Download the chromosome size files and make a table with the following information per species (Recall the N50 length is the length such that 50% of the genome is in chromosomes of this size or larger):

- Question 2.1. Total genome size

- Question 2.2. Number of chromosomes
- Question 2.3. Largest chromosome size and name
- Question 2.4. Smallest chromosome size and name
- Question 2.5. Mean chromosome length
- Question 2.6. N50 chromosome length

[72]: *#AI Disclosure: Used an LLM to generate the actual table to display*

```
#Function to compute n50 size given size array
def compute_n50(sizes):
    sorted_sizes = sorted(sizes, reverse=True)
    total = sum(sorted_sizes)
    half = total / 2
    counter = 0
    for size in sorted_sizes:
        counter += size
        if counter >= half:
            return size
    return None

#From file extracts names and sizes of chromosomes
def parse_file(filepath):
    chrom_names = []
    chrom_sizes = []
    with open(filepath) as f:
        for line in f:
            line = line.strip()
            if not line:
                continue
            parts = line.split()
            if len(parts) < 2:
                continue
            name, size = parts[0], parts[1]
            chrom_names.append(name)
            chrom_sizes.append(int(size))
    return chrom_names, chrom_sizes

#Gets stats from file path
def summarize_species(filepath):
    chrom_names, chrom_sizes = parse_file(filepath)

    total_size = sum(chrom_sizes)
    num_chroms = len(chrom_sizes)

    max_idx = chrom_sizes.index(max(chrom_sizes))
```

```

min_idx = chrom_sizes.index(min(chrom_sizes))

largest_size = chrom_sizes[max_idx]
largest_name = chrom_names[max_idx]

smallest_size = chrom_sizes[min_idx]
smallest_name = chrom_names[min_idx]

mean_length = total_size / num_chroms if num_chroms else 0
n50 = compute_n50(chrom_sizes)

# Use filename (without extension) as species identifier
species_name = filepath.split('/')[ -1 ].rsplit('.', 1)[0]

return {
    'Species': species_name,
    'Total Genome Size': total_size,
    'Number of Chromosomes': num_chroms,
    'Largest Chromosome': largest_name,
    'Largest Chromosome Size': largest_size,
    'Smallest Chromosome': smallest_name,
    'Smallest Chromosome Size': smallest_size,
    'Mean Chromosome Length': round(mean_length, 2),
    'N50': n50
}

import glob, os, pandas as pd, matplotlib.pyplot as plt
os.chdir("/Users/ashrithkollu/Documents/Classes/Junior_Fall/appliedgenomics2026/
↳ assignments/assignment1/")

files = glob.glob("*.sizes")

results = [summarize_species(f) for f in files]
df = pd.DataFrame(results)

print(df.to_string(index=False))

# Render as a matplotlib table
fig, ax = plt.subplots(figsize=(1.5 * len(df.columns), 0.6 * (len(df) + 1)))
ax.axis('off')

table = ax.table(
    cellText=df.values,
    colLabels=df.columns,
    cellLoc='center',
    loc='center'
)

```

```

table.auto_set_font_size(False)
table.set_fontsize(9)
table.auto_set_column_width(col=list(range(len(df.columns))))

# Style header row
for col_idx in range(len(df.columns)):
    header_cell = table[0, col_idx]
    header_cell.set_facecolor('#4C72B0')
    header_cell.set_text_props(color='white', weight='bold')

plt.title("Genome Assembly Summary", fontsize=12, weight='bold', pad=20)
plt.tight_layout()
plt.show()

```

Species	Total Genome Size	Number of Chromosomes	Largest Chromosome
Largest Chromosome Size	Smallest Chromosome	Smallest Chromosome Size	Mean
Chromosome Length	N50		
ecoli.chrom	4639211	1	Ecoli
4639211	Ecoli	4639211	4639211.00
4639211			
TAIR10.chrom	119146348	5	Chr1
30427671	Chr4	18585056	23829269.60
23459830			
tomato.chrom	782520033	13	ch01
90863682	ch00	9643250	60193848.69
65269487			
ce10.chrom	100286070	7	chrV
20924149	chrM	13794	14326581.43
17493793			
wheat.chrom	14547261565	22	3B
830829764	6D	473592718	661239162.05
709773743			
hg38.chrom	3088269832	24	chr1
248956422	chr21	46709983	128677909.67
156040895			
dm6.chrom	137547960	7	chr3R
32079331	chr4	1348131	19649708.57
25286936			
yeast.chrom	12157105	17	chrIV
1531933	chrM	85779	715123.82
924431			

Genome Assembly Summary

Species	Total Genome Size	Number of Chromosomes	Largest Chromosome	Largest Chromosome Size	Smallest Chromosome	Smallest Chromosome Size	Mean Chromosome Length	N50
ecoli.chrom	4639211	1	Ecoli	4639211	Ecoli	4639211	4639211.0	4639211
TAIR10.chrom	119146348	5	Chr1	30427671	Chr4	18585056	23829269.6	23459830
tomato.chrom	78252033	13	ch01	90863682	ch00	9643230	60193848.69	65269487
ce10.chrom	100286070	7	chrV	20924149	chrM	13794	14326581.43	17493793
wheat.chrom	14547261565	22	3B	830829764	6D	473592718	661239162.05	709773743
hg38.chrom	3088269832	24	chr1	248956422	chr21	46709983	128677909.67	156040895
dm6.chrom	137547960	7	chr3R	32079331	chr4	1348131	19649708.57	25286936
yeast.chrom	12157105	17	chrV	1531933	chrM	85779	715123.82	924431

0.0.4 Question 3. Random DNA shearing and fragment lengths

Before sequencing, DNA is often broken into smaller fragments. In this question, you will simulate randomly cutting a 1Mbp genome and examine the lengths of the fragments that remain. You do not need to simulate the DNA sequence itself.

- Question 3.1. In the language of your choice, simulate cutting a linear 1Mbp genome into 10,000 segments by picking 9,999 distinct cut points uniformly at random from positions 1 through 999,999. A cut at position x is between bases x and $x+1$. Sort the cut points, include the two ends of the genome, and compute the length of each remaining fragment. Report the mean fragment length. Plot a histogram of the fragment lengths and overlay an exponential distribution using the mean you just computed ($\lambda = 1/\text{mean}$). How well does the exponential distribution fit the data (just a visual comparison is enough)?

[61]: *#AI Disclosure: An LLM was used to generate the code for the overlay of the exponential distribution*

```
import random as rng
import numpy as np
import matplotlib.pyplot as plt

num_segs = 10000

def fragment_histogram(genomesize, num_segments):
    ## sample distinct cut positions without replacement
    cut_points = rng.sample(range(1, genomesize), num_segments-1)
    boundaries = np.concatenate([[0], np.sort(cut_points), [genomesize]])
    fragment_lengths = []

    for i in range(num_segments):
        length = boundaries[i+1] - boundaries[i]
        fragment_lengths.append(length)

    mean_length = np.mean(fragment_lengths)
    print(f"The mean fragment length is {mean_length:.3f}")
```

```

#Sim Histogram
plt.hist(fragment_lengths, bins='auto', orientation='vertical',
↪density='True')
plt.xlabel("Fragment length (in bp)")
plt.ylabel("Fragment Probability Density")
plt.title("Histogram of DNA Shearing Fragment Lengths")

#Overlay Expo
x = np.linspace(0, np.max(fragment_lengths), 1000)
pdf = 1/mean_length * np.exp(-1/mean_length * x)
plt.plot(x, pdf, 'r-', linewidth=2, label=f'Exponential fit,
↪(mean={mean_length:.1f})')

plt.legend()
plt.show()

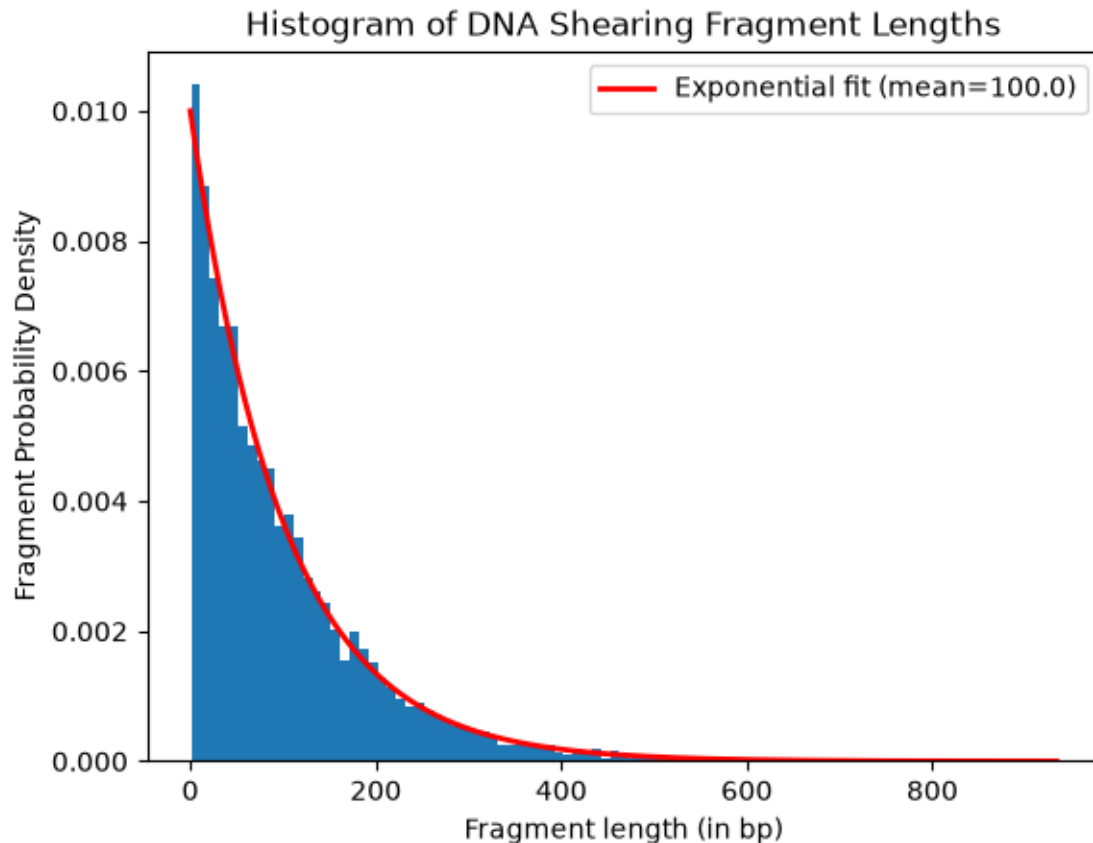
return fragment_lengths

fragment_lens = fragment_histogram(1000000, num_segs)

#The exponential distribution appears to fit the data quite well, with the
↪histogram edges nearly lining up perfectly with the overlaid exponential
↪distribution plot. The histogram appears a little jagged, though. This is
↪especially true near the left edge, with the smaller fragment sizes.

```

The mean fragment length is 100.000



- Question 3.2. Keep the fragments in their original order along the genome and divide them into non-overlapping groups of 10: fragments 1-10, 11-20, 21-30, etc. Compute the total length of each group, giving 1,000 observations. Plot a histogram of these total lengths and fit a negative binomial distribution using $r = 10$ and $p = 10$ divided by the observed mean group length. Overlay the fitted distribution. How well does it fit (just a visual comparison is enough)? How does the shape of this histogram compare with the histogram of individual fragment lengths?

[]: *#AI Disclosure: An LLM was used to generate the code necessary to create a*
↪negative binomial distribution overlay

```
from scipy.stats import nbinom
```

```
num_segs = 10000
```

```
def group_histogram(fragment_lengths_arr, num_segments):
```

```
    fragment_lengths_arr = np.array(fragment_lengths_arr)
```

```
    combined_lengths = []
```

```
    for i in range(num_segments//10):
```

```
        combined_lengths.append(sum(fragment_lengths_arr[i*10:i*10+10]))
```

```

mean_combined_length = np.mean(combined_lengths)

#Sim Histogram
plt.hist(combined_lengths, bins='auto', orientation='vertical',
density='True')
plt.xlabel("Group length (in bp)")
plt.ylabel("Group Probability Density")
plt.title(f"Histogram of Fragment Group Lengths, {num_segments} Segments")

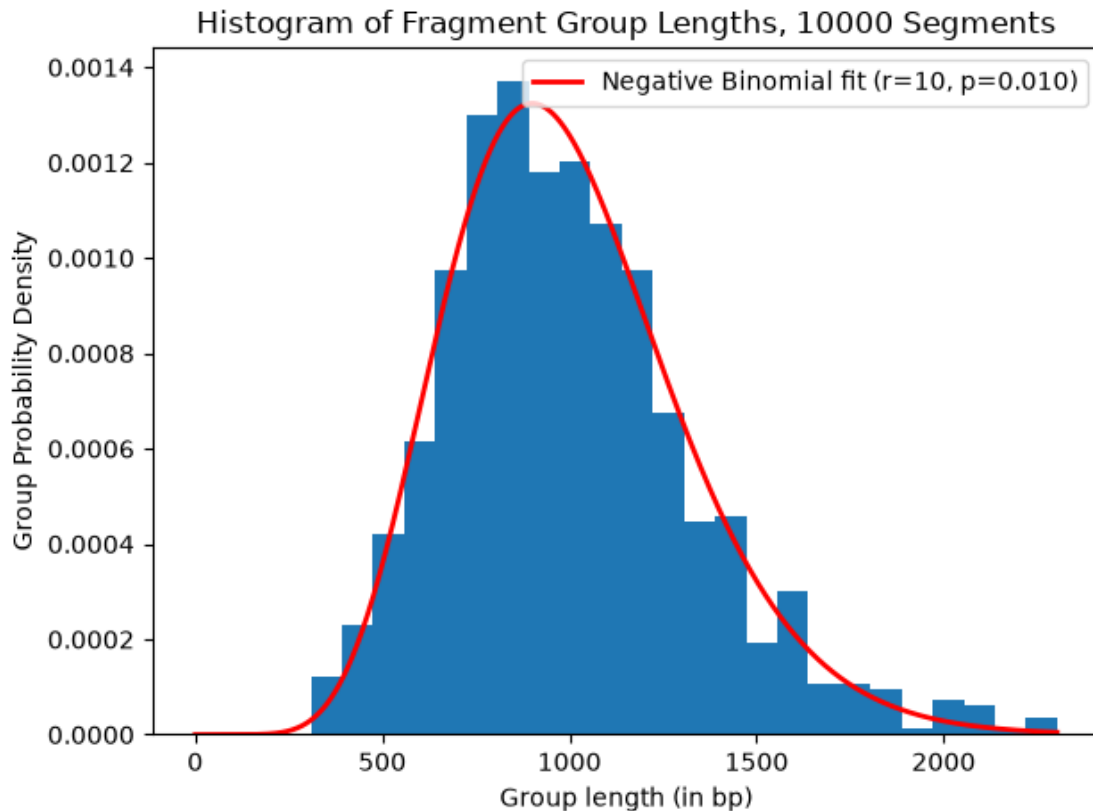
#Overlay Negative Binomial
r = 10
p = 10 / mean_combined_length
s = np.arange(0, np.max(combined_lengths)+1)
pmf = nbinom.pmf(s - 10, 10, p)
plt.plot(s, pmf, 'r-', linewidth=2, label=f'Negative Binomial fit (r={10},
p={p:.3f})')

plt.tight_layout()
plt.legend()
plt.show()

group_histogram(fragment_lens, num_segs)

#The overlay appears to roughly follow the shape of the histogram, suggesting
that a negative-binomial distribution is a good fit for the simulated data.
This histogram has a more bell-shaped curve compared to the monotonic
exponential decay curve from the previous histogram of fragment lengths.
However, it looks very jagged, which could be due to the large range of
group lengths plotted.

```



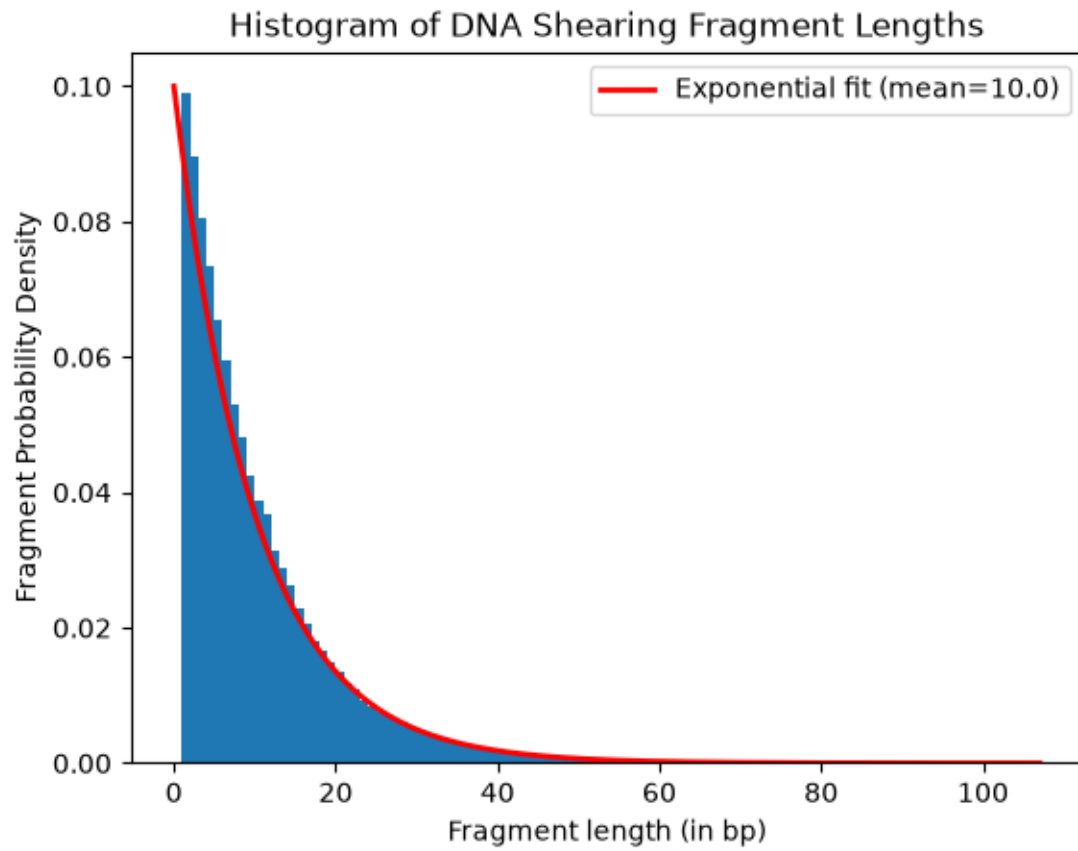
- Question 3.3. Repeat Questions 3.1 and 3.2, this time cutting the same 1Mbp genome into 100,000 segments. Report the mean fragment length and the mean total length of a group of 10. Show both histograms with their fitted distributions, using the new means to set the parameters. How do the lengths and the quality of the fits compare with your results for 10,000 segments?

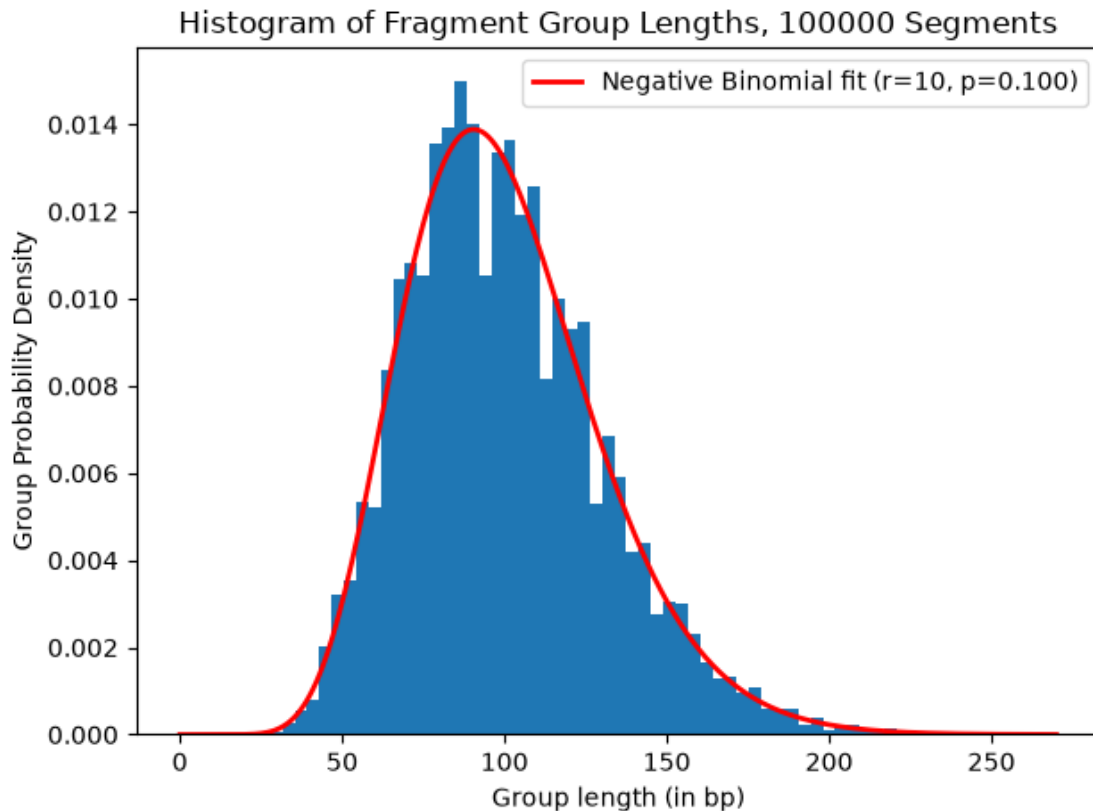
```
[ ]: num_segments2 = 100000

fragment_lens = fragment_histogram(1000000, num_segments2)
group_histogram(fragment_lens, num_segments2)

#The lengths differ significantly between the 10,000 segment simulation and
↳ this 100,000 segment simulation. Whereas the 10,000 segment simulation had
↳ fragment sizes ranging from 1 to around 500 for the most part, the 100,000
↳ segment simulation only had fragments from 1 to about 50 (visible on the
↳ histogram). This makes sense because there are 10 times as many cuts. The
↳ group lengths tell a similar tale, with the ranges seeming to differ by a
↳ factor of 10 as well. The quality of fit appears to be better with 100,000
↳ segments than 10,000, evident with the closer fitting overlaid lines.
```

The mean fragment length is 10.000





- Question 3.4. How many cuts should you make in a linear 1Mbp genome so that the mean fragment length is as close as possible to 42bp? Remember that the number of fragments is one more than the number of cuts. Report the integer number of cuts and the resulting mean fragment length.

```
[5]: print(f"The number of required cuts is 23809 and the resulting mean fragment_
↪length is {1000000/23810:.4f}")
```

The number of required cuts is 23809 and the resulting mean fragment length is 41.9992

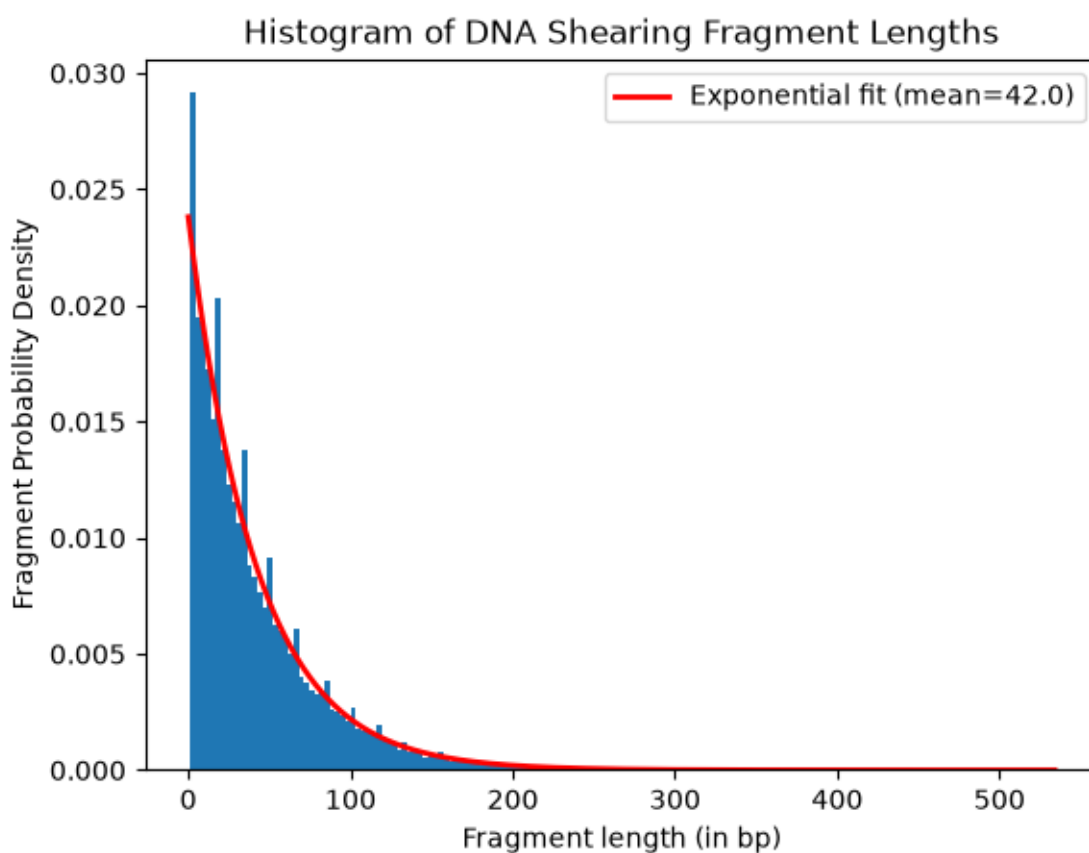
- Question 3.5. Run the simulator from Question 3.1 using the number of cuts you calculated in Question 3.4. Report the mean fragment length and show both histograms: individual fragment lengths with a fitted exponential distribution, and total lengths of non-overlapping groups of 10 with a fitted negative binomial distribution. If fewer than 10 fragments remain at the end, omit those fragments from the grouped histogram only. How well do the fitted distributions describe your results?

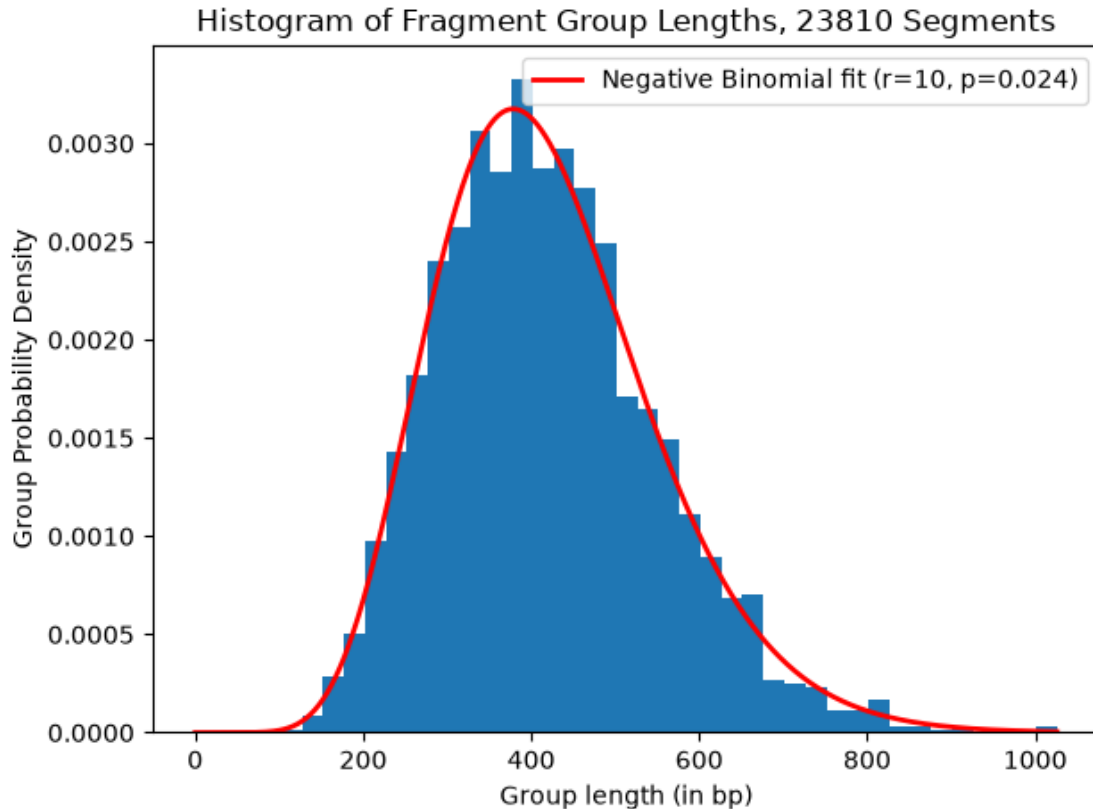
```
[69]: num_segments2 = 23810

fragment_lens = fragment_histogram(1000000, num_segments2)
group_histogram(fragment_lens, num_segments2)
```

#The fitted distributions describe the results quite well. The mean of the fragments is exactly what we expected: 42. With the exception of a couple of peaks above, the histogram generally appears to follow the shape of the overlaid exponential line well. The shape of the negative binomial is also close to the overlaid line.

The mean fragment length is 41.999





0.0.5 Question 4. Coverage simulator

- Question 4.1. How many 100bp reads are needed to sequence a 1Mbp genome to 3x coverage?

Answer: You would need $3 \times 1\text{Mbp}$ worth of data, which is $3000000/100 = 30000$. We need 30,000 reads to achieve 3x coverage.

- Question 4.2. In the language of your choice, simulate sequencing 3x coverage of a 1Mbp genome with 100bp reads and plot the histogram of coverage. Note you do not need to actually output the sequences of the reads, you can just uniform randomly sample positions in the genome and record the coverage. You do not need to consider the strand of each read. The start position of each read should have a uniform random probability at each possible starting position (1 through 999,901). You can record the coverage in an array of 1M positions. Overlay the histogram with a Poisson distribution with $\lambda=3$. Also overlay the distribution with a Normal distribution with a mean of 3 and a standard deviation of 1.73 (which is the square root of 3).

```
[7]: import random as rng
import scipy.stats
import matplotlib.pyplot as plt
import numpy as np
```

```

def run_coverage_sim(genomesize, coverage, readlength):

    num_reads = int(genomesize * coverage / readlength)

    read_positions = rng.choices(range(0, genomesize-readlength+1), k=num_reads)

    ## use an array to keep track of the coverage at each position in the genome
    genome_coverage = [0] * genomesize

    for i in range(num_reads):

        startpos = read_positions[i]
        endpos = startpos + readlength

        for j in range(startpos, endpos):
            genome_coverage[j] = genome_coverage[j] + 1

    maxcoverage = max(genome_coverage)

    ## use an array count how many positions have 0x coverage, have 1x coverage, ↵
    ↵have 2x coverage, ...
    histogram = [0] * (maxcoverage + 1)

    for k in range(genomesize):
        cov = genome_coverage[k]
        histogram[cov] = histogram[cov] + 1

    plt.bar(list(range(maxcoverage + 1)), histogram)
    plt.xlabel("x Coverage")
    plt.ylabel("Frequency")
    plt.title(f"Coverage and Frequency with {num_reads} reads of length ↵
    ↵{readlength} bp")

    mean = coverage

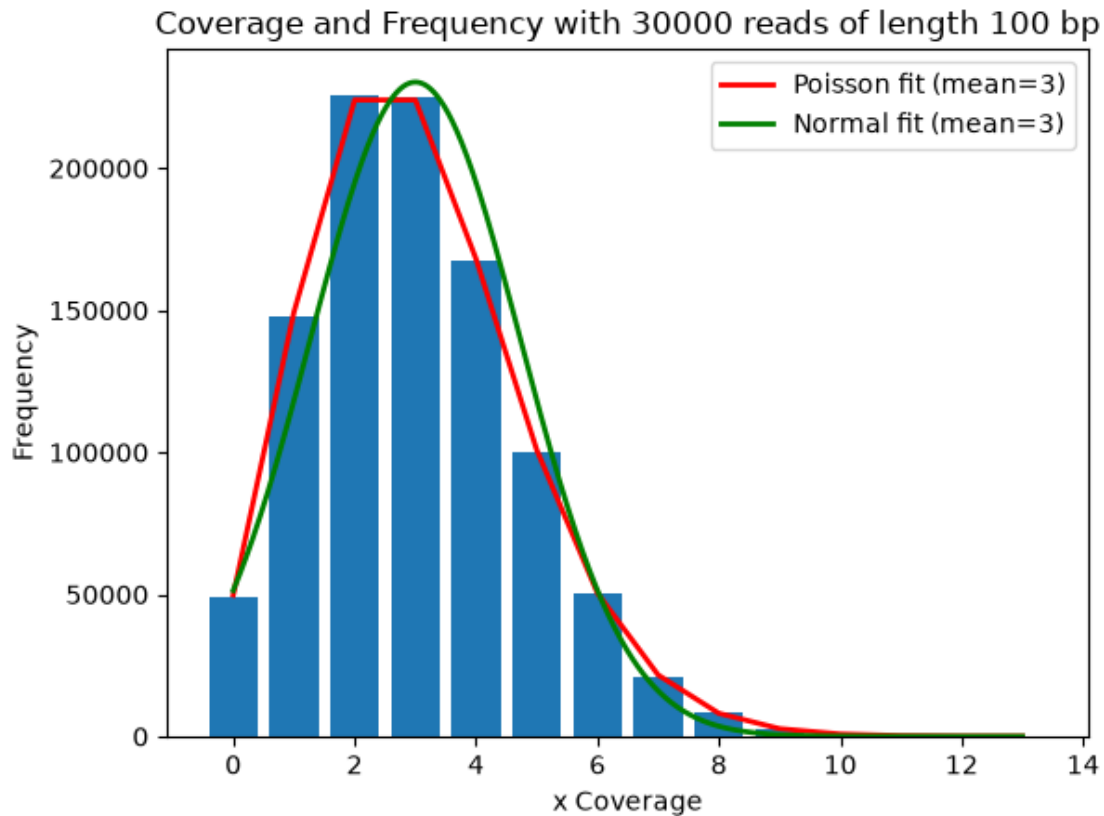
    x = np.arange(0, maxcoverage + 1)
    xcont = np.linspace(0, maxcoverage, 1000)
    pmf = scipy.stats.poisson.pmf(x, mean) * genomesize
    plt.plot(x, pmf, 'r-', linewidth=2, label=f'Poisson fit (mean={mean})')
    pdf = scipy.stats.norm.pdf(xcont, loc=mean, scale = np.sqrt(mean)) * ↵
    ↵genomesize
    plt.plot(xcont, pdf, 'g-', linewidth=2, label=f'Normal fit (mean={mean})')
    plt.legend()
    plt.show()

    return histogram

```



```
histogram = run_coverage_sim(1000000, 3, 100)
```



- Question 4.3. Using the histogram from Q4.2, how much of the genome has not been sequenced (has 0x coverage)? How well does this match Poisson expectations? How well does the normal distribution fit the data?

```
[8]: mean = 3
genome_size = 1000000

print(f"{histogram[0]} bps has not been sequenced (has 0x coverage). This
corresponds to a proportion of the genome of {histogram[0]/genome_size}.")

print(f"The poisson expectation for 0x coverage is {scipy.stats.poisson.pmf(0,
mean) * genome_size:.0f} base pairs, corresponding to a proportion of {scipy.
stats.poisson.pmf(0, mean):.4f}.")

print(f"The normal expectation for 0x coverage is {(scipy.stats.norm.cdf(0.5,
loc=mean, scale=np.sqrt(mean)) - scipy.stats.norm.cdf(-0.5, loc=mean,
scale=np.sqrt(mean))) * genome_size:.0f} base pairs, corresponding to a
proportion of {scipy.stats.norm.cdf(0.5, loc=mean, scale=np.sqrt(mean)) -
scipy.stats.norm.cdf(-0.5, loc=mean, scale=np.sqrt(mean)):.4f}.")
```

```
#The histogram matches the poisson expectations extremely well, differing only  
→by about 2%. The normal distribution does not match as well, yielding a  
→difference of about 8%. However, this, too, isn't terrible.
```

49356 bps has not been sequenced (has 0x coverage). This corresponds to a proportion of the genome of 0.049356.

The poisson expectation for 0x coverage is 49787 base pairs, corresponding to a proportion of 0.0498.

The normal expectation for 0x coverage is 52803 base pairs, corresponding to a proportion of 0.0528.

- Question 4.4. Now repeat the analysis with 10x coverage: 1. simulate the appropriate number of reads, 2. make a histogram, 3. overlay a Poisson distribution with lambda=10, 4. overlay with a Normal distribution with mean=10, standard deviation=3.16 (sqrt(10)). 5. compute the number of bases with 0x coverage, and 6. evaluate how well it matches the Poisson expectation and Normal expectations.

```
[9]: histogram = run_coverage_sim(1000000, 10, 100)

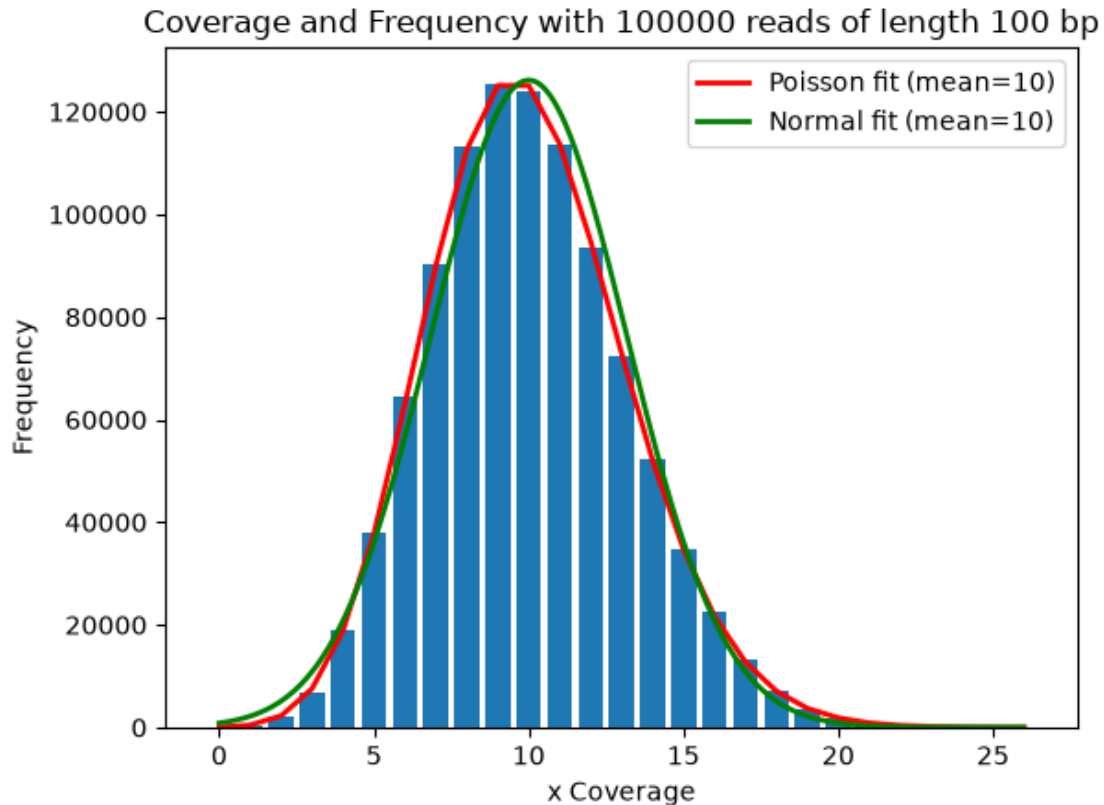
mean = 10
genomesize = 1000000

print(f"{histogram[0]} bps has not been sequenced (has 0x coverage). This  
→corresponds to a proportion of the genome of {histogram[0]/genomesize}.")

print(f"The poisson expectation for 0x coverage is {scipy.stats.poisson.pmf(0,  
→mean) * genomesize:.0f} base pairs, corresponding to a proportion of {scipy.  
→stats.poisson.pmf(0, mean):.4f}.")

print(f"The normal expectation for 0x coverage is {(scipy.stats.norm.cdf(0.5,  
→loc=mean, scale=np.sqrt(mean)) - scipy.stats.norm.cdf(-0.5, loc=mean,  
→scale=np.sqrt(mean))) * genomesize:.0f} base pairs, corresponding to a  
→proportion of {scipy.stats.norm.cdf(0.5, loc=mean, scale=np.sqrt(mean)) -  
→scipy.stats.norm.cdf(-0.5, loc=mean, scale=np.sqrt(mean)):.4f}.")

#The simulation matches the Poisson expectation extraordinarily well, with a  
→difference of just a couple dozen base pairs. The curve also follows the  
→histogram shape with amazing precision. The normal expectation also follows  
→the shape of the graph well, but has poorer approximation at the edges. The  
→expectation differs by a few hundred base pairs, which is still a decent  
→approximation considering it's only 0.09% of the genome.
```



63 bps has not been sequenced (has 0x coverage). This corresponds to a proportion of the genome of $6.3e-05$.

The poisson expectation for 0x coverage is 45 base pairs, corresponding to a proportion of 0.0000.

The normal expectation for 0x coverage is 882 base pairs, corresponding to a proportion of 0.0009.

- Question 4.5. Now repeat the analysis with 30x coverage: 1. simulate the appropriate number of reads, 2. make a histogram, 3. overlay a Poisson distribution with $\lambda=30$, 4. overlay with a Normal distribution with mean=30, standard deviation=5.47 ($\sqrt{30}$). 5. compute the number of bases with 0x coverage, and 6. evaluate how well it matches the Poisson expectation and Normal expectations.

```
[10]: histogram = run_coverage_sim(1000000, 30, 100)

mean = 30
genomesize = 1000000

print(f"{histogram[0]} bps has not been sequenced (has 0x coverage). This
↳ corresponds to a proportion of the genome of {histogram[0]/genomesize}.")
```

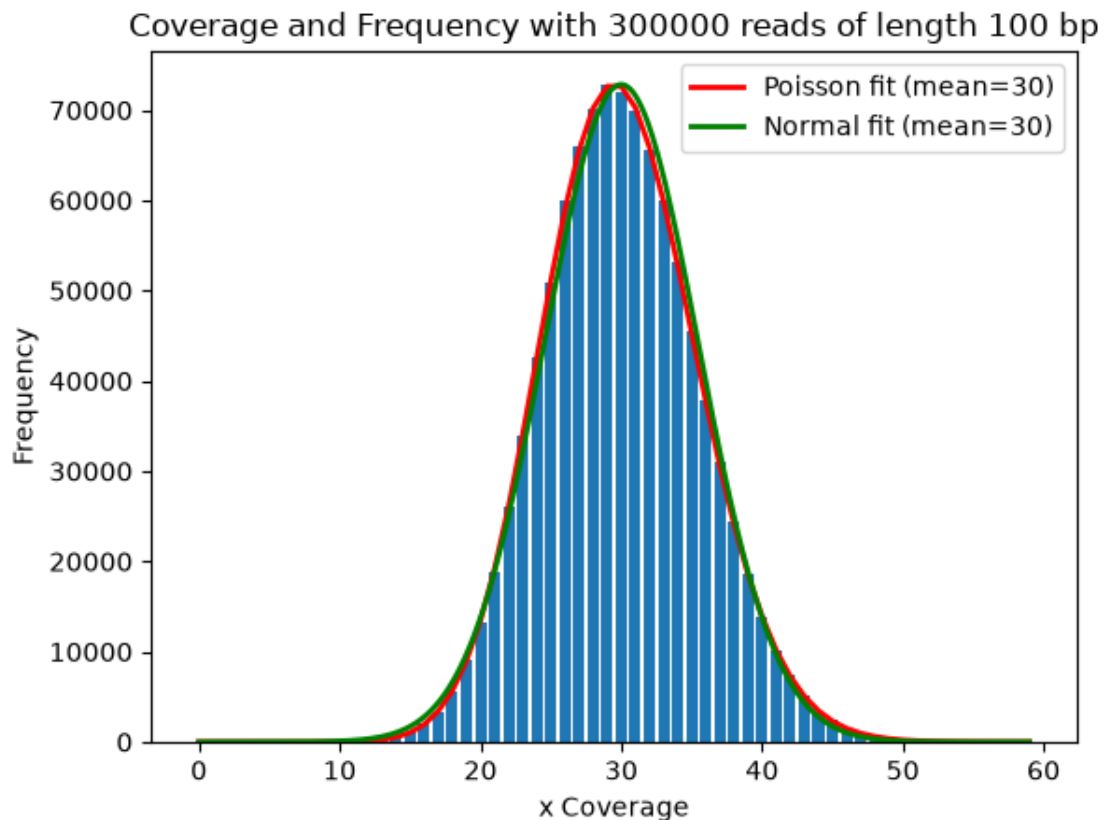
```

print(f"The poisson expectation for 0x coverage is {scipy.stats.poisson.pmf(0,
↳mean) * genomesize:.0f} base pairs, corresponding to a proportion of {scipy.
↳stats.poisson.pmf(0, mean):.8f}.")

print(f"The normal expectation for 0x coverage is {(scipy.stats.norm.cdf(0.5,
↳loc=mean, scale=np.sqrt(mean)) - scipy.stats.norm.cdf(-0.5, loc=mean,
↳scale=np.sqrt(mean))) * genomesize:.0f} base pairs, corresponding to a
↳proportion of {scipy.stats.norm.cdf(0.5, loc=mean, scale=np.sqrt(mean)) -
↳scipy.stats.norm.cdf(-0.5, loc=mean, scale=np.sqrt(mean)):.8f}.")

#The simulation matches the Poisson expectation almost perfectly, with a
↳difference of less than a dozen base pairs. The curve also follows the
↳histogram shape with near-perfect precision. The normal expectation also
↳follows the shape of the graph nearly perfectly. Other than a slight
↳right-shift of the peak and a slightly larger left tail, the normal
↳distribution curve is nearly identical to that of the Poisson distribution.
↳Consequently, the normal and poisson distributions have an identical
↳expected count for 0x coverage bases, which is 0.

```



2 bps has not been sequenced (has 0x coverage). This corresponds to a proportion of the genome of $2e-06$.

The poisson expectation for 0x coverage is 0 base pairs, corresponding to a proportion of 0.00000000.

The normal expectation for 0x coverage is 0 base pairs, corresponding to a proportion of 0.00000002.

0.0.6 Question 5: Kmer Uniqueness

- Question 5.1. How many As, Cs, Gs, Ts are found in the entire chromosome? What other characters are found and how often are they found?

```
[16]: fasta_dict = {}

file = open("/Users/ashrithkollu/Documents/Classes/Junior_Fall/CG-ACG/chr22.fa")

file.readline() #throw out the first line containing the name of the file

for line in file:

    line = line.strip()
    for c in line:
        if not c in fasta_dict:
            fasta_dict[c] = 1
        else:
            fasta_dict[c] = fasta_dict[c] + 1

for letter in ['A', 'T', 'C', 'G']:
    print(f"There are {fasta_dict[letter]} {letter}s in the chromosome.")

for letter in fasta_dict:
    if letter not in ['A', 'T', 'C', 'G']:
        print(f"There are also {fasta_dict[letter]} {letter}s in the_
↪chromosome.")

file.close()
```

There are 4583339 As in the chromosome.

There are 4582999 Ts in the chromosome.

There are 4524380 Cs in the chromosome.

There are 4521583 Gs in the chromosome.

There are also 11658691 Ns in the chromosome.

There are also 4724603 gs in the chromosome.

There are also 4636272 cs in the chromosome.

There are also 5798875 as in the chromosome.

There are also 5787726 ts in the chromosome.

- Question 5.2 Write a script to clean a fasta file by ensuring all characters are in uppercase (“a” becomes “A”) and replacing any non-ACGT characters with “A” (especially N characters). Run this script on chr22 and report how many As, Cs, Gs, and Ts are now found.

```
[24]: fasta_dict = {'A': 0,
                    'G' : 0,
                    'T' : 0,
                    'C' : 0}

file = open("/Users/ashrithkollu/Documents/Classes/Junior_Fall/CG-ACG/chr22.fa")
file_clean = open("/Users/ashrithkollu/Documents/Classes/Junior_Fall/CG-ACG/
↳chr22_clean.fa", 'w')

file.readline() #throw out the first line containing the name of the file

for line in file:

    line = line.strip().upper()
    clean_line = []

    for c in line:
        if not c in fasta_dict:
            fasta_dict['A'] += 1
            clean_line.append('A')
        else:
            fasta_dict[c] = fasta_dict[c] + 1
            clean_line.append(c)

    clean_line.append('\n')

    file_clean.write(''.join(clean_line))

for letter in ['A', 'T', 'C', 'G']:
    print(f"There are {fasta_dict[letter]} {letter}s in the chromosome.")

file.close()
file_clean.close()
```

There are 22040905 As in the chromosome.

There are 10370725 Ts in the chromosome.

There are 9160652 Cs in the chromosome.

There are 9246186 Gs in the chromosome.

- Question 5.3. In the language of your choice, tally the frequency of 19-mers in the cleaned chromosome file, and output the kmer frequency spectrum to a file e.g. how many kmers occur 1 time, how many occur 2 times, how many occur 3 times, etc. We recommend you use a dictionary (or hash table) to tally the frequencies using this pseudocode. In your writeup, show the kmer frequency spectrum for 1 to 20, e.g. how many kmers occur 1 time, how many occur 2, ..., how many occur 20 times. Note you should record all values in the file (up to max_frequency), but in your write up only display the first 20 (e.g. with the unix head command).

```

[74]: # initialize kmer length
k = 19

#open file
file = open("/Users/ashrithkollu/Documents/Classes/Junior_Fall/CG-ACG/
↳chr22_clean.fa")
output_file = open("/Users/ashrithkollu/Documents/Classes/Junior_Fall/CG-ACG/
↳chr22_kmerspectrum.txt", 'w')

# dictionary that maps a kmer (like GAT) to a frequency (like 3)
kmer_frequency = {}

## now scan the genome, extract kmers, and tally up their frequencies
genome = "".join(line.strip() for line in file)

for i in range(len(genome) - k + 1):
    kmer = genome[i : i + k]

    if kmer in kmer_frequency:
        kmer_frequency[kmer] = kmer_frequency[kmer] + 1
    else:
        kmer_frequency[kmer] = 1

## now tally the frequencies in a dictionary that maps kmer frequency to count
## also determine the maximum kmer frequency

tally = {}
all_kmers = kmer_frequency.keys()
max_frequency = 0

for kmer in all_kmers:

    freq = kmer_frequency[kmer]

    if freq in tally:
        tally[freq] = tally[freq] + 1
    else:
        tally[freq] = 1

    if (freq > max_frequency):
        max_frequency = freq

## now print in sorted order
for i in range(max_frequency + 1):

    if i in tally:
        freq = tally[i]

```

```

        output_file.write(f"{i} : {freq}\n")

file.close()
output_file.close()

```

```

[80]: import pandas as pd
df = pd.DataFrame(sorted(list(tally.items())) , columns=["K-mer Frequency",
↪ "Number of K-mers"])
print(df.head(20))

```

	K-mer Frequency	Number of K-mers
0	1	29426883
1	2	989900
2	3	264955
3	4	123280
4	5	57189
5	6	32993
6	7	23782
7	8	17025
8	9	13269
9	10	10841
10	11	9079
11	12	7525
12	13	6495
13	14	5724
14	15	4830
15	16	4388
16	17	3979
17	18	3462
18	19	3095
19	20	2806

- Question 5.4. Using the output from 5.3, plot the kmer frequency spectrum: x-axis is the kmer frequency, and the y-axis is the number of kmers that occur x times. Make sure to plot both the x and y-axis in log space. This should include all kmers (up to max_frequency)

```

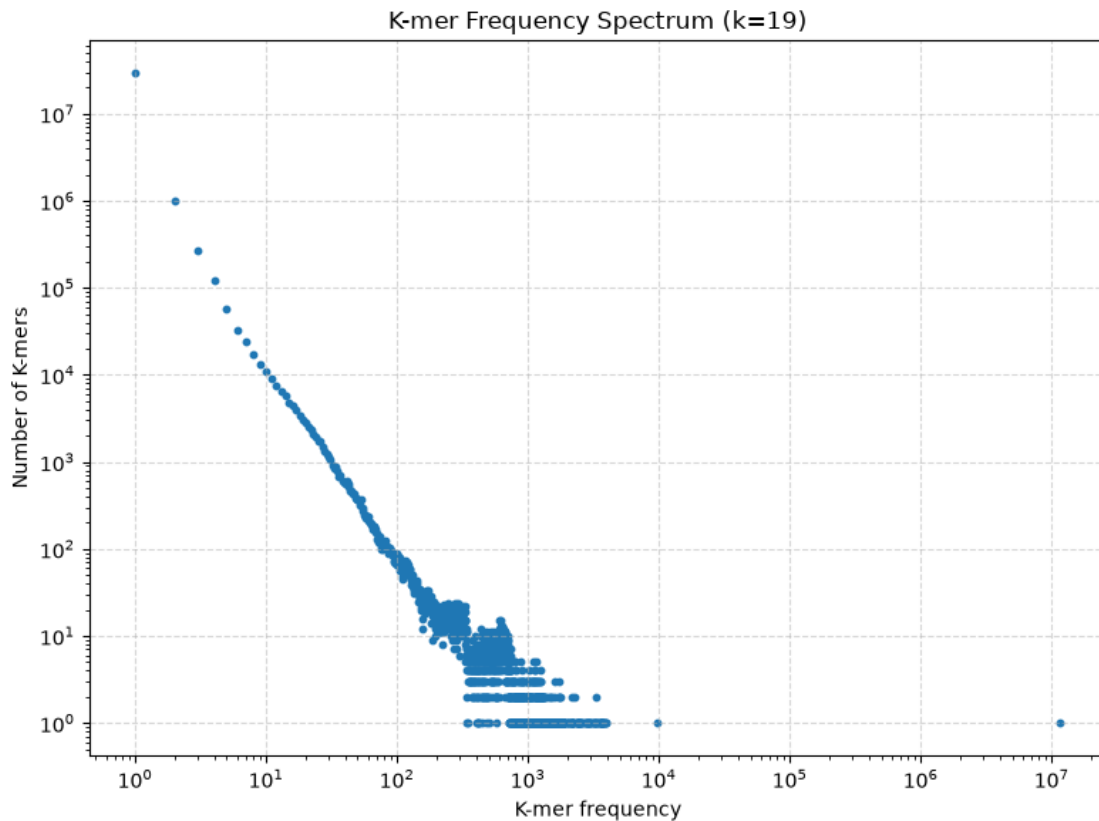
[32]: frequencies = sorted(tally.keys())
counts = [tally[f] for f in frequencies]

plt.figure(figsize=(8, 6))
plt.scatter(frequencies, counts, s=10)
plt.xscale('log')
plt.yscale('log')
plt.xlabel('K-mer frequency')
plt.ylabel('Number of K-mers')
plt.title('K-mer Frequency Spectrum (k=19)')
plt.grid(True, which='major', linestyle='--', alpha=0.5)
plt.tight_layout()

```



```
plt.show()
```



- Question 5.5. a) What percent of the genome is unique, e.g. what percent of the kmers occur 1 time. b) What percent of the genome is repetitive (occurs more than 1 time). c) What percent occurs 1000 or more times?

```
[71]: total_kmers = sum(fasta_dict.values()) - k + 1

print(f"The percent of the genome that is unique is {tally[1]/total_kmers * 100_
↳:.4f}%.")
print(f"The percent of the genome that is repetitive is {(total_kmers -
↳tally[1])/total_kmers * 100 :.4f}%.")

thousandmore = 0

for i in tally:
    if (i >= 1000):
        thousandmore += i * tally[i]

print(f"The percent of the genome that occurs 1000 or more times is_
↳{thousandmore/total_kmers * 100 :.4f}%.")
```

The percent of the genome that is unique is 57.9059%.
The percent of the genome that is repetitive is 42.0941%.
The percent of the genome that occurs 1000 or more times is 24.4457%.